



THE OS FOR MULTI-AGENT DEVELOPMENT

synlynk

quick start.

From zero to a context-aware, cost-tracked,
multi-agent workgroup in under two minutes.

v0.4.1 Instruction Reach

```
Terminal

$ synlynk init

✓ Project context injected
✓ 3 agents discovered · claude · gemini · codex
✓ 7 instruction files tracked in .synlynk/instructions.json
✓ Budget guard active · $10.00 / session
✓ Sentinel monitoring & drift detection enabled

→ synlynk instructions status
```

Zero dependencies

Python 3 stdlib only

Claude · Gemini · Codex

Cost tracking

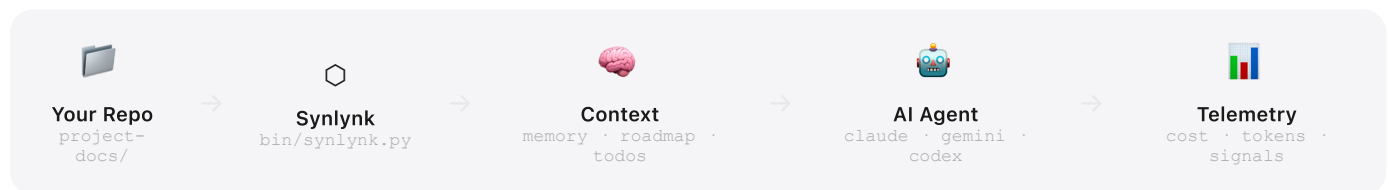
Sentinel guard

Background dispatch

WHAT IS SYNLYNK?

Your AI agents, working together.

Synlynk is a single-file Python CLI that wraps AI agents — Claude, Gemini, Codex — and makes them project-aware, cost-tracked, and safe to run in parallel. One file. Zero dependencies. Works anywhere Python 3 runs.



Why Synlynk?



Context-Aware by Default

Every agent invocation automatically receives your roadmap, memory, and todos. No more copy-pasting context into every session.



Zero Surprise Bills

Budget guard blocks execution the moment you hit your daily or session limit. Real-time cost tracking on every exec.



Hallucination Sentinel

Automatically detects flatline loops, success loops, and quota exhaustion. Alerts before damage is done.



Parallel Dispatch

Launch Claude, Gemini, and Codex simultaneously on different tasks. Monitor all jobs from one CLI while your terminal stays free.



Team-Ready

Single or team mode. Per-user devlogs with git attribution. Shared roadmap and memory visible to every agent in the repo.



Zero Dependencies

Pure Python 3 stdlib. No pip install, no Docker, no build step. One file — works in any environment where Python 3 runs.

1

FILE — NO BUILD STEP

3

AI AGENTS SUPPORTED

0

PIP DEPENDENCIES

∞

PARALLEL AGENT JOBS

GETTING STARTED

Up and running in 60 seconds.

1 Clone & install globally

Adds `synlynk` to your PATH via `~/.synlynk/bin/`. No sudo, no pip, no build step required.

```
install

$ git clone https://github.com/nikhilsoman/synlynk
$ cd synlynk && ./install.sh

✓ Installed to ~/.synlynk/bin/synlynk
✓ PATH updated in ~/.zshrc
✓ synlynk 0.4.1 ready
```

2 Bootstrap any project

Run inside any repo. Creates `project-docs/` and writes `CLAUDE.md`, `GEMINI.md`, `AI_INSTRUCTIONS.md`. Safe to re-run — skips existing files.

```
synlynk init

~/my-project $ synlynk init

Mode: solo | team? solo

✓ project-docs/roadmap.md · memory.md · todo.md · costs.md
✓ CLAUDE.md · GEMINI.md · AGENTS.md · AI_INSTRUCTIONS.md
✓ Tracked 7 instruction files in .synlynk/instructions.json
✓ Agents: claude · gemini · codex

→ synlynk instructions status
```

3 Run your first context-injected session

Synlynk prepends your project context — the AI already knows your roadmap, decisions, and open tasks before you type a word.

```
synlynk exec

$ synlynk exec claude

📅 Context injected (memory · roadmap · todos)
💰 Budget: $2.34 / $10.00 used this session
🚀 Launching claude...

claude > I can see you're building a billing API for the
monetization module. Based on your roadmap...
```

• Magic Moments

The init wizard scans your README, git history, and existing docs to pre-fill your roadmap and memory with real context — not generic placeholders.

V0.4.1 · INSTRUCTION REACH

Run agents in parallel.

In the background.

Dispatch fires an agent as a background job — captured to a log file, tracked by the job store. Multiple agents run simultaneously while your terminal stays free.

```
synlynk dispatch

$ synlynk dispatch claude --task "Write unit tests for the billing module"
✓ Dispatched job-a3f2 (claude) running in background

$ synlynk dispatch gemini --task "Review billing API for edge cases"
✓ Dispatched job-b7e1 (gemini) running in background

# Both agents working simultaneously — terminal is yours
```

Monitor all jobs

```
synlynk jobs

$ synlynk jobs

• job-a3f2 claude running 2m14s
• job-b7e1 gemini running 1m58s
```

Tail a job's output

```
synlynk logs

$ synlynk logs job-a3f2

Writing test_billing.py...
✓ test_charge_success
✓ test_refund_partial
→ 12 / 12 tests passing
```

The Trio Pipeline v0.4.1

Three agents in sequence — Architect designs, Builder implements, Verifier reviews — with full context hand-off between each phase.

```
synlynk run --trio

$ synlynk run --trio "Add Stripe webhook handler"

Phase 1 Architect (claude) ... ✓ done job-cla0
Phase 2 Builder (gemini) ... ✓ done job-d4b2
Phase 3 Verifier (claude) ... ✓ done job-e9f3

✓ Trio complete. synlynk logs job-e9f3
```

- **Coming in v0.5.0 — Smart Routing**

Dispatch will automatically route to the best-performing agent for each task domain, based on historical capability scores. Pass `--story <id>` now to start tracking work items.

SENTINEL & BUDGET GUARD

Your safety net, always on.

Synlynk watches every agent invocation for danger signals and surfaces alerts before they become expensive mistakes.

Budget Guard

Set spending limits in `.synlynk/config.json`. Synlynk blocks the next exec the moment you'd exceed your limit. Cost shown after every run.

```
.synlynk/config.json
{
  "limit_usd": 10.00,
  "limit_requests": 100
}

budget pulse

— Budget Pulse —
Cost:      $2.34 / $10.00
Requests: 14 / 100
Tokens:    2,840 in · 480 out
```

Sentinel Patterns

Sentinel watches your telemetry log and fires alerts automatically for five danger patterns.

- FLATLINE** **Same command failed 3x in a row**
Possible hallucination loop. Intervene before more tokens burn.
- LOOP** **Same command succeeded 5x in <10 min**
Likely an automated loop running unproductively. Investigate.
- QUOTA** **Quota exhaustion detected in output**
Output matched a known quota-exhausted phrase. Switch agent.
- ZOMBIE** **Job PID dead or stalled >30 min**
Reconciled automatically on the next run.
- DRIIFT** **Instruction file edited outside synlynk**
SHA mismatch on tracked files. Use [instructions diff/update](#).

Managing Alerts

```
sentinel commands

$ synlynk sentinel list

[CRITICAL] FLATLINE      `synlynk exec claude` failed 3x — 2026-06-14 09:41
[WARN]    SUCCESS_LOOP  Same command 5x in 4.2 min — 2026-06-14 10:02

$ synlynk sentinel clear --code FLATLINE
✓ Cleared FLATLINE alerts
```

ALL COMMANDS

Everything at a glance.

Setup & Context

COMMAND	WHAT IT DOES
<code>synlynk init</code>	Bootstrap <code>project-docs/</code> , agent instruction files, and <code>.synlynk/config.json</code> in any repo. Safe to re-run — skips existing files.
<code>synlynk exec <agent></code>	Run an agent interactively with full context injection (memory + roadmap + todos prepended). Captures token counts and cost on exit.
<code>synlynk checkpoint</code>	Force a context snapshot — regenerates <code>.synlynk/context.md</code> from current project-docs state.
<code>synlynk status</code>	Dashboard showing current project state, recent costs, active sentinel alerts, and running jobs.
<code>synlynk upgrade</code>	Check for and apply updates from the remote repository.

Dispatch & Jobs v0.4.1

COMMAND	WHAT IT DOES
<code>synlynk dispatch <agent></code>	Fire background task. Captures stdout. Accepts <code>--task "..."</code> and <code>--story <id></code> .
<code>synlynk jobs</code>	List tracked jobs (running / completed / failed) and duration. <code>--all</code> shows completed.
<code>synlynk logs <job-id></code>	Tail output log of a job. Use <code>--tail N</code> to control lines.
<code>synlynk run --trio</code>	Run Architect → Builder → Verifier pipeline with context hand-off.

Sentinel & Instructions v0.4.1

COMMAND	WHAT IT DOES
<code>synlynk sentinel list / clear</code>	Show all active alerts / clear targeted alerts via <code>--severity</code> or <code>--code</code> .
<code>synlynk instructions status</code>	Show tracking status of all registered instruction files (CLAUDE.md, etc.).
<code>synlynk instructions diff [file]</code>	Show user edits/custom changes outside the synlynk section for review.
<code>synlynk instructions update [file]</code>	Reset the synlynk section to current template and refresh manifest SHA.
<code>synlynk instructions ack <file></code>	Acknowledge DRIFT alert and clear it from the sentinel alerts log.

Project Docs Layout

```
project-docs/
├── roadmap.md      ← version plan
├── memory.md       ← decisions
├── todo.md         ← open tasks
├── costs.md        ← cost ledger
├── devlogs/
│   └── nikhil.md   ← per-user log
└── .synlynk/
    ├── config.json ← budget limits
    ├── context.md  ← auto-generated
    ├── instructions.json ← tracked file SHAs
    ├── telemetry.json ← rolling log
    └── logs/        ← job outputs
```

What agents receive

Every `synlynk exec` prepends `memory.md` + `roadmap.md` + `todo.md` as a single context snapshot before handing off to the agent.

● Keep project-docs current

Synlynk's context is only as good as your `memory.md` and `roadmap.md`. Update them after significant decisions — agents pick up changes on the next exec automatically.

● Use it in every repo

`synlynk init` + `synlynk exec` are safe to use in any project immediately. Hold off on `dispatch` in critical workflows until v0.7.0 daemon lands in October.

WHAT'S COMING NEXT

The road to v1.0.

Synlynk evolves from an agent wrapper into a full OS for multi-agent development. Each release adds a new layer of the stack.

v0.5 **Capability Engine** ✓ Live · June 2026
Model-aware routing · SQLite WAL state ledger · capability rating & story management

v0.6 **Job Control + Constraints** ✓ Live · June 2026
Constraint propagation · job state machine · PR validation check · model attestation

v0.7 **Async Pipeline + Daemon** Planned · Oct 2026
Background daemon · HTTP context server · synlynk review TUI

v0.8 **Open Context Protocol** Planned · Nov 2026
context --for · checkpoint --from · MCP server · ecosystem interface

v1.0 **Stable OS + Tokq Ready** Planned · Q1 2027
Frozen CLI · pipx/Homebrew distribution · NATS leaf node schema defined